The Big Bull Query

A SQL Story of Power, Greed and Markets

Bombay. 1980.

A boy from Raipur with **40** in his pocket steps onto Dalal Street.

He has no degree. No contacts. No capital.

He has only one thing — the ability to *see patterns where others see noise.*

Twelve years later, he would move the entire Indian stock market with a phone call.

He would live in a **15,000 sq ft penthouse** with a swimming pool, a billiard room and a fleet of Lexus cars — while millions of Indians had never heard of a stock exchange.

He would meet the Prime Minister of India. He would be on the front page of every newspaper. He would be called **The Big Bull** — the man who made the Sensex roar.

And then, on **April 23, 1992**, a single newspaper article would take it all apart in a single morning.

His name was **Harshad Shantilal Mehta.**

And the truth about him is locked inside the data.

How to Play

The year is **1992**. The CBI is raiding offices. Parliament is in chaos. The Sensex has crashed **40%**. Thousands of investors have lost everything.

You are the investigator.

Every chapter gives you a set of database tables — some filled with real historical records, some padded with noise. Your job is to write SQL queries that cut through the clutter, find the real connections, and uncover what actually happened.

The further you dig, the more uncomfortable the answers get.

- Was Harshad a criminal?
- Or was he a genius the system used — and then destroyed?
- Who really escaped?
- What did the Prime Minister know?

The data has the answers. Write the query. Find the truth.

8 Chapters. 40+ Tables. 100+ SQL Puzzles.

From Raipur to Rajya Sabha — one query at a time.

Chapter 1

The Boy from Raipur

SQL Concepts Covered: `SELECT`, `WHERE`, `AND/OR`, `ORDER BY`, `COUNT`, `DISTINCT`, `BETWEEN`, `LIKE`, `IS NOT NULL`

Question 1 **Intro Level**

"It is 1954. Somewhere in India, a boy is born who will one day shake the entire stock market. Let's find him first."

Task: Select all details of the main character.

Hint: Use `is_main_character` to find him.

Question 2 **Intro Level**

"Harshad was born on a specific date that he was proud of. But many records have been tampered with. Find only those records that match his exact birth date."

Task: Find all characters born on July 29, 1954.

Hint: Filter using `birth_date`.

Question 3 **Easy**

"Not everyone born on that date is our man. Harshad came from a Gujarati

family, born in Raipur. His father's name was Shantilal Mehta. Can you find the exact match?"

Task: Find the character born on July 29, 1954 in Raipur, from the Gujarati community, whose father is Shantilal Mehta.

Hint: Use `AND` to combine multiple conditions.

Question 4 — Easy

"Harshad had brothers. His family was the engine behind his ambition. Find his real family members — not the noise."

Task: From the `family` table, find all real family members of `character_id = 1`, ordered by relation.

Hint: Filter `is_real = TRUE` and `character_id = 1`.

Question 5 — Easy

"The Mehta family left Raipur and came to Bombay chasing a dream. But when exactly did they move, and where did they settle in Bombay?"

Task: Find all real cities Harshad (`character_id = 1`) actually lived in, ordered by the year he moved there.

Hint: Filter `is_real = TRUE`, order by `moved_year`.

Question 6 — Easy

"Harshad grew up in Kandivali, Bombay — a chawl boy in a city that didn't care about your past, only your hustle. Which schools and colleges did he actually attend?"

Task: Find all real education records of `character_id = 1`, ordered by `start_year`.

Hint: Filter `is_real = TRUE` and `character_id = 1`.

Question 7 **Medium**

"After B.Com, Harshad had no godfather, no connections, no money. He took whatever work he could find. How many real jobs did he do before entering the stock market?"

Task: Count the number of real jobs held by `character_id = 1`.

Hint: Use `COUNT` with `WHERE` filters.

Question 8 **Medium**

"From hosiery to diamonds to cement — Harshad tried it all. List all his real jobs with salary, from lowest paid to highest paid."

Task: Find all real jobs of `character_id = 1`, ordered by `monthly_salary` ascending.

Hint: Use `ORDER BY monthly_salary ASC`.

Question 9 **Medium**

"Before becoming the Big Bull, Harshad was just another man surviving Bombay. What was his total earning capacity across all his real jobs combined per month?"

Task: Find the `SUM` of `monthly_salary` for all real jobs of `character_id = 1`.

Hint: Use the `SUM()` aggregate function.

Question 10 **Medium**

"The turning point was near. Harshad's last real job before the stock market was as a Sub-Broker Assistant. Find that job — the one that changed everything."

Task: Find the job of `character_id = 1` where `job_title` contains the word `Broker`.

Hint: Use `LIKE` with wildcard `%`.

Question 11 **Medium**

"By 1979, Harshad had already tried multiple jobs. Find all real jobs he held between 1977 and 1980 — the years of grinding before the breakthrough."

Task: Find real jobs of `character_id = 1` where `start_year` is between 1977 and 1980.

Hint: Use `BETWEEN`.

Question 12 **Hard**

"Someone is trying to forge records and replace Harshad's identity. Find all characters who share his birth date AND birth city — potential impostors in the database."

Task: Find all characters with `birth_date = '1954-07-29'` AND `birth_city = 'Raipur'`, showing `full_name` and `father_name` to spot differences.

Hint: Use `AND` conditions; compare `father_name` carefully.

Question 13 **Hard**

"Harshad's two real brothers — Ashwin and Sudhir — were part of his business empire later. But the family table has noise. Find only the male family members (brothers) who are real."

Task: Find all real family members of `character_id = 1` whose relation is `'Brother'`.

Hint: Filter `relation = 'Brother'` and `is_real = TRUE`.

Question 14 **Hard**

"The records show Harshad lived in many cities — but most of them are planted lies. Show the distinct real states he actually lived in."

Task: Find `DISTINCT` states from `cities_lived` where `character_id = 1` and `is_real = TRUE`.

Hint: Use `SELECT DISTINCT`.

Question 15

Chapter Boss — Hard

"You have now traced the full early life of Harshad Mehta. Build his complete profile — his real birth details, his real final education, and the job that first introduced him to the stock market world."

Task: Write a query that shows:

- `full_name`, `birth_date`, `birth_city` — from `characters`
- His highest education: `institution`, `degree` — from `education` (highest `end_year` among real records)
- His last real job before the stock market: `job_title`, `employer` — from `early_jobs` (highest `start_year` among real records)

Hint: Use subqueries with `MAX()` to find the most recent education and job.

Chapter 2

The BSE Floor

SQL Concepts Covered: `SELECT`, `WHERE`, `AND/OR`, `JOIN`, `LEFT JOIN`, `GROUP BY`, `HAVING`, `COUNT`, `SUM`, `MAX`, `ORDER BY`, `DISTINCT`

Question 1 **Intro Level**

"It is 1980. Bombay's BSE floor is a jungle of numbers, screaming brokers, and chalk boards. Harshad walks in as nobody. Find his official BSE member record."

Task: Find the BSE member record where `character_id = 1`.

Hint: Simple `SELECT` with `WHERE`.

Question 2 **Intro Level**

"The BSE has hundreds of brokers. How many of them registered in the same year as Harshad — 1984? He wasn't alone, but he was different."

Task: Count all BSE members who registered in 1984.

Hint: Use `COUNT` with `WHERE`.

Question 3 **Easy**

"Many people share the Mehta surname on the BSE floor. Find all active BSE

members whose last name contains 'Mehta' — Harshad needs to know his competition."

Task: Find all active `bse_members` whose `full_name` contains `'Mehta'`, ordered by `registration_year`.

Hint: Use `LIKE` and `is_active = TRUE`.

Question 4 — Easy

"Harshad was not born a Full Member. He started as a Sub-Broker. Find all Sub-Brokers who were active in Bombay during 1980 to 1984 — the era when Harshad was grinding his way up."

Task: Find all active Sub-Brokers in Bombay who registered between 1980 and 1984.

Hint: Use `BETWEEN` and `AND` conditions.

Question 5 — Easy

"Before Growmore, Harshad learned everything from one man on the BSE floor. Find his real mentor — the person who truly taught him the game."

Task: Find all real mentor records for `character_id = 1` from the `mentors` table.

Hint: Filter `is_real = TRUE` and `character_id = 1`.

Question 6 — Easy

"In 1984, Harshad did the most important thing of his life — he registered his own firm. Find the real firm he founded, not the noise."

Task: Find all real firms where `character_id = 1`.

Hint: Filter `is_real = TRUE` and `character_id = 1`.

Question 7 — Medium

"On the BSE floor, Harshad traded aggressively from day one. Find all his real trades and calculate the total profit or loss he made across all of them."

Task: Find the `SUM` of `profit_loss` for all trades where `is_harshad = TRUE`.

Hint: Use `SUM()` aggregate.

Question 8 — Medium

"Harshad had a sixth sense for certain stocks. Which stock did he trade the most number of times in his early career?"

Task: From `stock_trades_early` where `is_harshad = TRUE`, `GROUP BY stock_name` and `COUNT` trades, ordered by count descending.

Hint: Use `GROUP BY` with `COUNT`, `ORDER BY DESC`.

Question 9 — Medium

"Not every trade was a winner. Even the Big Bull had bad days. Find all Harshad's real trades where he made a loss — the ones he never talked about."

Task: Find all trades where `is_harshad = TRUE` and `profit_loss < 0`, ordered by `profit_loss ASC`.

Hint: Filter `profit_loss < 0`.

Question 10 — Medium

"Harshad's best year in this early phase was 1986 — the year everything clicked. How much total profit did he make just in 1986?"

Task: Find `SUM` of `profit_loss` for `is_harshad = TRUE` and `trade_year = 1986`.

Hint: Add `trade_year` filter to your `WHERE` clause.

Question 11 — Medium

"Year by year, Harshad was getting bigger. Show his total profit for each year

from 1984 to 1986 so we can see his growth trajectory."

Task: `GROUP BY trade_year` for `is_harshad = TRUE`, show `SUM(profit_loss)` per year, ordered by year.

Hint: `GROUP BY trade_year`, `ORDER BY trade_year ASC`.

Question 12 — Medium

"Harshad's life changed on key dates. Find all his real personal milestones ordered by year — the real story of how he rose in this chapter."

Task: Find all real personal milestones for `character_id = 1`, ordered by `event_year`.

Hint: Filter `is_real = TRUE`, `character_id = 1`.

Question 13 — Hard

"The BSE floor had many cities but Bombay was the heart. Find each city and count how many Full Members were active there — but only show cities that had MORE than 5 active Full Members."

Task: `GROUP BY city` for Full Members where `is_active = TRUE`, use `HAVING COUNT(*) > 5`.

Hint: Use `GROUP BY` + `HAVING`.

Question 14 — Hard

"Harshad's firm Growmore needed a BSE membership to operate. Join the firms table with bse_members to show Harshad's firm details alongside his BSE registration details."

Task: `JOIN firms` and `bse_members` on `character_id`, filter where `firms.character_id = 1` and `firms.is_real = TRUE`.

Hint: Use `INNER JOIN` on `character_id`.

Question 15

Hard

"A rival broker claims he made more total profit than Harshad in 1985. Find the top 3 broker IDs by total profit in 1985 — and see where Harshad ranks."

Task: `GROUP BY broker_id` for `trade_year = 1985`, `SUM(profit_loss)`, `ORDER BY total DESC`, `LIMIT 3`.

Hint: Use `GROUP BY broker_id`, `ORDER BY DESC`, `LIMIT 3`.

Question 16

Chapter Boss — Hard

"You have traced Harshad's full Chapter 2 arc. Now build his complete BSE profile — his member details, his real firm, his mentor, and his total profit earned in this chapter."

Task: Write a query that shows:

- `full_name`, `member_type`, `registration_year` — from `bse_members` where `character_id = 1`
- `firm_name` — from `firms` (real, `character_id = 1`, earliest `registration_year`)
- `mentor_name` — from `mentors` (real, `character_id = 1`)
- `total_profit` — from `stock_trades_early` where `is_harshad = TRUE`

Hint: Use multiple `JOIN`s with subqueries.

Chapter 3

The Money Machine

SQL Concepts Covered: `SELECT`, `WHERE`, `JOIN`, `LEFT JOIN`, `GROUP BY`, `HAVING`, `CASE WHEN`, `SUM`, `AVG`, `ROUND`, *Subqueries*

Question 1 **Intro Level**

"The scam was not built overnight. It was built on a network of banks. Find all the banks that were actually involved in Harshad's scheme."

Task: Find all banks where `was_involved = TRUE`.

Hint: Simple `SELECT WHERE`.

Question 2 **Intro Level**

"Not all banks were equal in this game. Some were public, some were foreign. How many involved banks were there of each `bank_type`?"

Task: `GROUP BY bank_type` where `was_involved = TRUE`, `COUNT` each type.

Hint: `GROUP BY` + `COUNT`.

Question 3 **Easy**

"The Bank Receipt — the weapon of the scam. Harshad used fake BRs to siphon money. Find ALL fake bank receipts in the system."

Task: Find all records from `bank_receipts` where `is_fake = TRUE`.

Hint: `WHERE is_fake = TRUE`.

Question 4 — Easy

"How much total money did Harshad steal through fake Bank Receipts? Add up all the fake BR amounts — this is the number that shocked India."

Task: Find `SUM` of `amount_crore` where `is_fake = TRUE`.

Hint: `SUM()` with `WHERE` filter.

Question 5 — Easy

"National Housing Bank was the biggest weapon. Find all fake BRs that were issued specifically by NHB — `bank_id = 62`."

Task: Find fake BRs from `bank_id = 62`, ordered by `issued_date`.

Hint: `WHERE is_fake = TRUE AND bank_id = 62`.

Question 6 — Easy

"Ready Forward deals were the vehicle. All the fraudulent RF deals went into one place — the stock market. Find all fraudulent RF deals and where the money was diverted."

Task: Find all `rf_deals` where `is_fraudulent = TRUE`, show `deal_amount_cr` and `diverted_to`.

Hint: `WHERE is_fraudulent = TRUE`.

Question 7 — Medium

"The fraud grew bigger every month. Show the total fraudulent RF deal amount grouped by where the money was diverted — which stocks got the most fuel?"

Task: `GROUP BY diverted_to` for `is_fraudulent = TRUE`, `SUM deal_amount_cr`, `ORDER BY total DESC`.

Hint: `GROUP BY diverted_to` + `SUM`.

Question 8

Medium

"Harshad's real associates were the nerve center of his operation. Find all his real associates and what role they played."

Task: Find all associates where `character_id = 1` and `is_real = TRUE`, ordered by `year_connected`.

Hint: `WHERE` + `ORDER BY`.

Question 9

Medium

*"Harshad used `CASE WHEN` to categorize the banks in his mind — big guns vs small pawns. Classify each involved bank as **'Major Player'** if their `bank_type` is Public or Foreign, else **'Minor Player'**."*

Task: Use `CASE WHEN` to add a classification column for all banks where `was_involved = TRUE`.

Hint: `CASE WHEN bank_type IN (...) THEN ... ELSE ... END`

Question 10

Medium

"Among the real assets Harshad acquired, what was his total estimated wealth from real assets only? And what was his average asset value?"

Task: From `lifestyle` where `is_real = TRUE` and `character_id = 1`, find `COUNT`, `SUM` and `ROUND(AVG)` of `estimated_value`.

Hint: Use `COUNT`, `SUM`, `ROUND(AVG(...), 2)`.

Question 11

Medium

"Harshad's lifestyle was split across asset types. Group his real assets by `asset_type` and find the total value of each type."

Task: `GROUP BY asset_type` for `character_id = 1` and `is_real = TRUE`, `SUM estimated_value`.

Hint: `GROUP BY asset_type` + `SUM`.

Question 12 — Hard

"Join the fraudulent RF deals with the banks table to see which bank was responsible for the most fraudulent money flow."

Task: `JOIN rf_deals` and `banks` on `bank_id`, `WHERE is_fraudulent = TRUE`, `GROUP BY bank_name`, `SUM deal_amount_cr ORDER BY total DESC`.

Hint: `JOIN` + `GROUP BY` + `SUM`.

Question 13 — Hard

"Some banks had both fake BRs AND fraudulent RF deals. Find banks that appear in BOTH `bank_receipts` (`is_fake = TRUE`) AND `rf_deals` (`is_fraudulent = TRUE`)."

Task: Find `bank_ids` that exist in both tables using a subquery with `IN`.

Hint: `WHERE bank_id IN (SELECT bank_id FROM ...WHERE ...)`

Question 14 — Hard

"LEFT JOIN the banks to their fake BRs. Some involved banks have no fake BRs on record — they may have destroyed evidence. Find involved banks with NO fake BR records."

Task: `LEFT JOIN banks` with `bank_receipts` (`is_fake = TRUE`) on `bank_id`, `WHERE was_involved = TRUE` AND `br_id IS NULL`.

Hint: `LEFT JOIN` + `IS NULL` to find missing matches.

Question 15 — Hard

"Which bank contributed more than 500 crore in total fake BR amounts? These are the banks that truly powered the scam."

Task: `JOIN banks` and `bank_receipts` where `is_fake = TRUE`, `GROUP BY bank_name`, `HAVING SUM(amount_crore) > 500`.

Hint: `JOIN` + `GROUP BY` + `HAVING`.

Question 16

Chapter Boss — Hard

*"You have now cracked the Money Machine. Build the full fraud summary — for each involved bank show: bank name, total fake BRs, total fake BR amount, total fraudulent RF deals, total RF amount, and classify each bank using `CASE WHEN` as **'Primary Conspirator'** if total combined amount > 500 crore, else **'Secondary'**."*

Task: Write a query that shows:

- `bank_name`, `bank_type` — from `banks`
- `total_fake_brs` and `fake_br_total` — from `bank_receipts` (`is_fake = TRUE`)
- `total_rf_deals` and `rf_deal_total` — from `rf_deals` (`is_fraudulent = TRUE`)
- `combined_total_crore` — sum of both amounts
- Bank role using `CASE WHEN`: **'Primary Conspirator'** if combined total > 500, else **'Secondary Participant'**

Hint: Use two `JOIN`s + `GROUP BY` + `CASE WHEN` on `SUM`.

Chapter 4

The Big Bull

SQL Concepts Covered: `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING`, `JOIN`, `CASE WHEN`, Subqueries, CTEs, Window Functions (`RANK`, `SUM OVER`)

Level 1 — The Floor Whispers

Basic SELECT, WHERE, ORDER BY

Question 1 — Level 1

"The year is 1991. You just walked onto Dalal Street. Everyone is talking about one stock that's flying. Find all ACC Cement price records and order them from earliest to latest date. What you should see: the rocket journey from 200 to 9000."

Task: Find all records from `stock_prices` where `stock_name = 'ACC Cement'`, ordered by `price_date ASC`.

Hint: Simple `SELECT` with `WHERE` and `ORDER BY`.

Question 2 — Level 1

"A journalist stops you and asks: 'Which months was the Sensex actually in a bull run?' List all Sensex records where `is_bull_run` is TRUE, showing month, year and Sensex value."

Task: Find all records from `sensex_index` where `is_bull_run = TRUE`, showing `record_month`, `record_year` and `sensex_value`.

Hint: `WHERE is_bull_run = TRUE`, `ORDER BY record_year`.

Question 3 — Level 1

"You want to know the peak of the madness. Find the single highest closing price ACC Cement ever hit."

Task: Find `MAX(close_price)` from `stock_prices` where `stock_name = 'ACC Cement'`.

Hint: Use the `MAX()` aggregate function.

Question 4 — Level 1

"A retail investor grabs your collar and yells: 'I lost everything! How many people like me are in this database?' Count how many retail investors recorded a loss."

Task: Count records from `investor_activity` where `is_retail = TRUE` and `profit_loss < 0`.

Hint: `COUNT(*)` with two `WHERE` conditions.

Level 2 — Following the Money

Aggregations, GROUP BY, HAVING

Question 5 — Level 2

"The market is roaring. You want to understand which stocks retail investors piled into the most. Show each stock bought and the total amount invested across all retail investors, ordered by total invested DESC."

Task: From `investor_activity` where `is_retail = TRUE`, `GROUP BY stock_bought` and `SUM(amount_invested)`, ordered by total descending.

Hint: `GROUP BY` + `SUM` + `ORDER BY DESC`.

Question 6 — Level 2

"How much did the Sensex actually grow during the bull run? Calculate the difference between the highest and lowest `sensex_value` where `is_bull_run = TRUE`. This is the size of Harshad's manufactured boom."

Task: Find `MIN`, `MAX` and the difference (`MAX - MIN`) of `sensex_value` where `is_bull_run = TRUE`.

Hint: Use `MIN()` and `MAX()` in the same `SELECT`.

Question 7 — Level 2

"The press is writing about Harshad. Show each publication and how many articles they published, but only show publications that published MORE than 1 article. Also count positive vs negative tones."

Task: From `media_coverage`, `GROUP BY publication`, `COUNT` articles, use `CASE WHEN` for tone counts, `HAVING COUNT(*) > 1`, ordered by article count DESC.

Hint: `GROUP BY` + `HAVING` + `CASE WHEN` inside `SUM`.

Level 3 — The Web Tightens

JOINs across tables

Question 8 — Level 3

"The CBI officer hands you a file. It contains all stocks bought by retail investors AFTER the Sensex had already started crashing (after April 23, 1992). These are the people who bought at the peak — the real victims. Find their names, stock bought, entry date and loss amount."

Task: From `investor_activity`, find records where `entry_date > '1992-04-23'` and `profit_loss < 0`, ordered by `profit_loss ASC`.

Hint: Combine date filter and profit filter in `WHERE`.

Question 9 — Level 3

"The landmark article that broke everything was published on April 23, 1992 by Sucheta Dalal. On that exact same date, what happened to ACC Cement stock? JOIN `media_coverage` and `stock_prices` on the same date to show the article title alongside the stock crash numbers."

Task: `JOIN media_coverage` and `stock_prices` on `publish_date = price_date`, filter `is_landmark = TRUE` and `stock_name = 'ACC Cement'`, show price drop.

Hint: `JOIN` on matching date columns; compute `close_price - open_price` as `price_drop`.

Question 10 — Level 3

"A political analyst tells you: 'Not all meetings with politicians were real. Harshad only confirmed 2 meetings.' List only the CONFIRMED political meetings along with the politician name, their designation and the meeting outcome."

Task: From `political_meetings`, find all records where `is_confirmed = TRUE`, ordered by `meeting_year`.

Hint: Simple `WHERE is_confirmed = TRUE`.

Level 4 — Connecting the Conspiracy

Subqueries, CTEs, Window Functions

Question 11 — Level 4

"The JPC committee wants a month-by-month picture of the Sensex during the bull run, showing each month's RUNNING TOTAL gain from the start of the bull run. Use a window function to calculate the cumulative monthly change."

Task: From `sensex_index` where `is_bull_run = TRUE`, use `SUM(monthly_change) OVER (ORDER BY year, month)` to compute a cumulative gain column.

Hint: Use `SUM(...) OVER (ORDER BY ...)` as a window function; convert month names to numbers using `CASE WHEN` inside `ORDER BY`.

Question 12 — Level 4

"The investigator wants to rank all stocks by total money retail investors poured into them. Use `RANK()` to assign a rank and also show what percentage of total investment each stock captured."

Task: Use a CTE to `SUM(amount_invested)` per `stock_bought` for retail in-

vestors, then apply `RANK() OVER` and calculate each stock's percentage of the grand total using a `CROSS JOIN`.

Hint: Two CTEs — one for stock totals, one for grand total — then `RANK() OVER (ORDER BY total_invested DESC)` and `ROUND(total * 100.0 / grand_total, 2)`.

Question 13

Level 4

"Find the SINGLE WORST DAY for ACC Cement — the date with the largest single-day price drop. Show the date, open, close, and the drop amount. Use a subquery to find the maximum drop first."

Task: From `stock_prices` where `stock_name = 'ACC Cement'`, find the row where `(open_price - close_price)` equals the result of a subquery using `MAX(open_price - close_price)`.

Hint: `WHERE (open_price - close_price) = (SELECT MAX(...) FROM ...)`

Level 5 — The Final Reckoning

Multi-table CTEs, the story's verdict query

Question 14

The Verdict Query

"Build a complete picture of the April 23, 1992 crash day — the article that exposed Harshad, the ACC stock price on that day, the Sensex in April 1992, and the number of retail investors who entered AFTER the crash and still lost money. Combine all four as a single summary report using CTEs and cross joins."

Task: Write a query using four CTEs:

- `breaking_article` — Sucheta Dalal's landmark article from `media_coverage`
- `acc_crash` — ACC Cement price on `'1992-04-23'` from `stock_prices`
- `sensex_crash` — Sensex value for April 1992 from `sensex_index`
- `victims` — count of retail investors with `entry_date > '1992-04-23'` and `profit_loss < 0`

Then `CROSS JOIN` all four CTEs into a single summary row.

Hint: Define each CTE with `WITH ...AS (...)`, then `SELECT` from the first

and `CROSS JOIN` the remaining three.

Bonus — The Ghost in the Data

The question the game never asks but the player must find

Bonus Question

Unscripted

"Harshad always claimed the REAL culprit was the system — the bankers, the politicians, the brokers who all knew. Find all political meetings marked `is_confirmed = FALSE` where the purpose mentions 'protection' or 'political', AND find all media articles marked `is_landmark = TRUE` where the tone is 'Neutral' — the ones that didn't take a side. These are the grey zones of the story."

Task: Write a `UNION ALL` query combining:

- Unconfirmed political meetings from `political_meetings` where purpose contains `'protection'` or `'political'` (use `ILIKE`)
- Neutral landmark articles from `media_coverage` where `tone = 'Neutral'`

Label each row with a `record_type` column and order by `record_type`, `name`.

Hint: Use `UNION ALL` with matching column aliases; `ILIKE '%protection%'` for case-insensitive pattern matching.

Chapter 5

The Crash

SQL Concepts Covered: `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING`, `JOIN`, `BOOL_OR`, `CASE WHEN`, *Subqueries, CTEs, Window Functions (*`RANK`, `SUM OVER`, `FIRST_VALUE`*)*

Level 1 — The Morning After

Basic SELECT, WHERE, ORDER BY

Question 1 — Level 1

"April 23, 1992. The article is out. The CBI is moving fast. Find all REAL raids ordered by `raid_date` *— show what happened in the first 30 days after the crash."*

Task: From `cbi_raids`, find all records where `is_real = TRUE`, ordered by `raid_date ASC`.

Hint: Simple `SELECT WHERE` with `ORDER BY`.

Question 2 — Level 1

"The investor in front of you is crying. 'Which day was the absolute worst for the market?' Find the single black day with the highest `pct_fall` *from* `market_crash_impact`*."*

Task: From `market_crash_impact` where `is_black_day = TRUE`, find the record with the highest `pct_fall`. Show `impact_date`, `sensex_open`, `sensex_close`, `points_fallen`, `pct_fall` and `retail_panic_level`.

Hint: `ORDER BY pct_fall DESC LIMIT 1`.

Question 3 — Level 1

"How many banks suffered REAL losses that were NOT recoverable? Count them from `bank_losses`."

Task: Count records from `bank_losses` where `is_real = TRUE` and `recovery_possible = FALSE`.

Hint: `COUNT(*)` with two `WHERE` conditions.

Question 4 — Level 1

"List all real public reactions in order — show the story of those 10 weeks from April 23 to July 1992."

Task: From `public_reaction`, find all records where `is_documented = TRUE`, showing `reaction_date`, `reaction_type`, `source`, `description` and `sentiment`, ordered by `reaction_date ASC`.

Hint: `WHERE is_documented = TRUE ORDER BY reaction_date ASC`.

Level 2 — Counting the Damage

Aggregations, GROUP BY, HAVING

Question 5 — Level 2

"The RBI wants a total damage report. Sum up all REAL bank losses in crores. Show each `bank_id` with their total loss and order by worst loss first."

Task: From `bank_losses` where `is_real = TRUE`, `GROUP BY bank_id`, show `SUM(loss_amount_cr)`, `COUNT(*)` and `BOOL_OR(recovery_possible)`, ordered by total loss DESC.

Hint: `GROUP BY bank_id` + `SUM` + `ORDER BY DESC`.

Question 6 — Level 2

"How did public sentiment shift week by week? Group `public_reaction` by sentiment type and count how many documented reactions fell into each sentiment category."

Task: From `public_reaction` where `is_documented = TRUE`, `GROUP BY sentiment`, `COUNT(*)` each group, ordered by count DESC.

Hint: `GROUP BY sentiment` + `COUNT` + `ORDER BY DESC`.

Question 7 — Level 2

"The CBI wants to know which raids actually seized documents vs those that found cash. From real raids only, group by whether `documents_seized` is TRUE or FALSE and show the total `cash_seized_cr` per group."

Task: From `cbi_raids` where `is_real = TRUE`, `GROUP BY documents_seized`, show `COUNT(*)` and `SUM(cash_seized_cr)`.

Hint: `GROUP BY documents_seized` — a boolean column works as a group key.

Level 3 — The Evidence Chain

JOINs across tables

Question 8 — Level 3

"The court needs a joined evidence file. For every REAL CBI raid, show the raid details alongside ALL assets seized in that raid. JOIN `cbi_raids` and `seized_assets` on `raid_id`."

Task: `JOIN cbi_raids` and `seized_assets` on `raid_id`, filter `is_real = TRUE` on both tables, order by `raid_date` then `estimated_value DESC`.

Hint: Apply `is_real = TRUE` to both tables in the `WHERE` clause.

Question 9 — Level 3

"The JPC committee wants to correlate the crash days with documented public reactions. JOIN `market_crash_impact` and `public_reaction` where the dates match — show what happened on the streets the same day the market fell."

Task: `JOIN market_crash_impact` and `public_reaction` on `impact_date =`

`reaction_date`, filter `is_black_day = TRUE` and `is_documented = TRUE`, ordered by `impact_date`.

Hint: `JOIN` on matching date columns from both tables.

Question 10 — Level 3

"Connect `bank_losses` to the `banks` table from Chapter 3. Show each real bank's name, type and their total loss amount — only for banks that suffered real losses."

Task: `JOIN bank_losses` and `banks` on `bank_id`, filter `bl.is_real = TRUE`, `GROUP BY bank_name` and `bank_type`, `SUM(loss_amount_cr)`, ordered by total loss DESC.

Hint: `JOIN` + `GROUP BY` + `SUM` + `BOOL_OR`.

Level 4 — The Forensic Report

Subqueries, CTEs, Window Functions

Question 11 — Level 4

"Build a running total of market damage day by day. Use a window function on `market_crash_impact` to show cumulative points fallen across all black days, and the cumulative percentage lost, ordered by date."

Task: From `market_crash_impact` where `is_black_day = TRUE`, use `SUM(points_fallen) OVER (ORDER BY impact_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` for cumulative total. Also compute the cumulative percentage using `FIRST_VALUE(sensex_open) OVER (ORDER BY impact_date)`.

Hint: Two window functions in the same `SELECT` — one for cumulative sum, one using `FIRST_VALUE` as the denominator.

Question 12 — Level 4

"The CBI lawyer needs to rank the banks by how much they lost — and flag which ones can never recover the money. Use `RANK()` and a `CASE` label."

Task: Use a CTE (`bank_damage`) to `JOIN bank_losses` and `banks`, `GROUP BY bank_name` and `SUM` losses. Then apply `RANK() OVER (ORDER BY total_loss_cr`

DESC) and a `CASE WHEN any_recovery = FALSE` label.

Hint: CTE first, then window function and `CASE WHEN` in the outer `SELECT`.

Question 13 — Level 4

"Find all seized assets whose `estimated_value` is ABOVE the average estimated value of all real seized assets. These are the crown jewels the court froze."

Task: From `seized_assets` where `is_real = TRUE`, find records where `estimated_value >` a subquery computing `AVG(estimated_value)` from the same table, ordered by value DESC.

Hint: `WHERE estimated_value > (SELECT AVG(...) FROM seized_assets WHERE is_real = TRUE)`

Level 5 — The Final Reckoning

Multi-table CTE verdict query

Question 14 — The Crash Verdict Query

"Build a complete collapse summary using five angles: total real bank losses, total seized asset value, the worst single crash day, the first documented public reaction, and the arrest raid record. Five CTEs. One picture of the devastation."

Task: Write a query using five CTEs:

- `total_bank_loss` — `SUM(loss_amount_cr)` from `bank_losses` where `is_real = TRUE`
- `total_seized` — `SUM(estimated_value)` from `seized_assets` where `is_real = TRUE`
- `worst_day` — highest `pct_fall` black day from `market_crash_impact`
- `first_reaction` — earliest documented record from `public_reaction`
- `arrest_raid` — the arrest record from `cbi_raids` where `raid_type = 'Arrest'`

Then `CROSS JOIN` all five into a single summary row.

Hint: Define each CTE with `WITH ...AS (...)`, `SELECT` from the first and `CROSS JOIN` the remaining four.

Bonus — The Bombshell Press Conference

The moment Harshad named the Prime Minister

Bonus Question

Unscripted

"On May 10, 1992, Harshad held a press conference and claimed PM Narasimha Rao told him to boost the market. Find that public reaction record. Then cross it with the `political_meetings` table to show what the official record says about the same claim. Two versions of the same truth."

Task: Write a `UNION ALL` query combining:

- The `public_reaction` record for `reaction_date = '1992-05-10'` where `is_documented = TRUE`
- The `political_meetings` record for `politician_name = 'P.V. Narasimha Rao'` where `is_confirmed = TRUE`; use `MAKE_DATE(meeting_year, 1, 1)` for the date column and `CASE WHEN is_confirmed` for the tone column

Label each row with a `source_type` column and order by `source_type`.

Hint: `UNION ALL` requires both `SELECT`s to have the same number of columns with matching aliases. Use `MAKE_DATE()` to convert the year integer into a date.

Chapter 6

The Political Storm

SQL Concepts Covered: `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING`, `JOIN`, `CASE WHEN`, `ILIKE`, `INTERVAL`, Subqueries, CTEs, Window Functions (`COUNT OVER`, `FIRST_VALUE`, `LAST_VALUE`)

Level 1 — The Inquiry Begins

Basic SELECT, WHERE, ORDER BY

Question 1 — Level 1

"The JPC has just been formed. List all REAL witnesses who testified, ordered by hearing date — show who said what and whether they were hostile."

Task: From `jpc_hearings`, find all records where `is_real = TRUE`, showing `hearing_date`, `witness_name`, `witness_role`, `testimony_summary`, `was_hostile` and `evidence_submitted`, ordered by `hearing_date ASC`.

Hint: Simple `SELECT WHERE ORDER BY`.

Question 2 — Level 1

"The opposition is looking for every debate where a motion was actually PASSED. List all real parliament debates where `motion_passed = TRUE` — these are the moments that actually changed something."

Task: From `parliament_debates`, find all records where `is_real = TRUE` and `motion_passed = TRUE`, showing `debate_date`, `house`, `raised_by`, `party`, `subject`

and `govt_response`, ordered by `debate_date ASC`.

Hint: Two conditions in `WHERE` joined by `AND`.

Question 3 — Level 1

"Who actually got punished vs who walked free? Do a simple count from `scapegoats_vs_escaped` grouped by category."

Task: From `scapegoats_vs_escaped`, `GROUP BY category` and `COUNT(*)` each group, ordered by count DESC.

Hint: `GROUP BY category` + `COUNT` — no `WHERE` filter needed.

Question 4 — Level 1

"The RBI is under fire. List all REAL RBI actions ordered by date — show the story of how they scrambled to fix the system they failed to protect."

Task: From `rbi_actions`, find all records where `is_real = TRUE`, showing `action_date`, `action_type`, `description`, `triggered_by` and `banks_affected`, ordered by `action_date ASC`.

Hint: `WHERE is_real = TRUE ORDER BY action_date ASC`.

Level 2 — Follow the Power

Aggregations, GROUP BY, HAVING

Question 5 — Level 2

"Which political party raised the most real debates in parliament about the scam? Group `parliament_debates` by party where `is_real = TRUE` and count debates per party. Also show how many motions each party got passed."

Task: From `parliament_debates` where `is_real = TRUE`, `GROUP BY party`, show `COUNT(*)` as `debates_raised` and `SUM(CASE WHEN motion_passed ...)` as `motions_passed`, ordered by debates DESC.

Hint: `CASE WHEN motion_passed THEN 1 ELSE 0 END` inside `SUM`.

Question 6 — Level 2

"How many real JPC witnesses submitted evidence vs those who just testified verbally? Group by `evidence_submitted` from real hearings, and also count how many in each group were hostile."

Task: From `jpc_hearings` where `is_real = TRUE`, `GROUP BY evidence_submitted`, show `COUNT(*)` and `SUM(CASE WHEN was_hostile ...)` as `hostile_count`.

Hint: Boolean column as group key; `CASE WHEN` inside `SUM` for conditional count.

Question 7 — Level 2

"The investigative journalist wants to know: of all real people questioned, how many from each organisation managed to walk free, get charged, or get convicted?"

Task: From `scapegoats_vs_escaped`, `GROUP BY organisation`, show counts for `walked_free`, `was_charged` and `was_convicted` using `CASE WHEN` inside `SUM`, ordered by `walked_free DESC`.

Hint: Three separate `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` columns.

Level 3 — Connecting the Web

JOINs across tables

Question 8 — Level 3

"The JPC committee is cross-referencing witnesses with politicians questioned on the same dates. JOIN `jpc_hearings` and `politicians_questioned` where `hearing_date = question_date` to find days when both happened simultaneously."

Task: `JOIN jpc_hearings` and `politicians_questioned` on `hearing_date = question_date`, filter `is_real = TRUE` on both tables, ordered by date.

Hint: `JOIN` on matching date columns; apply `is_real` filter to both table aliases.

Question 9 — Level 3

"Connect parliament debates to RBI actions — find every case where an RBI action was taken within 30 days AFTER a parliament debate. These are the debates that actually forced the RBI's hand."

Task: `JOIN parliament_debates` and `rbi_actions` where `ra.action_date BETWEEN pd.debate_date AND pd.debate_date + INTERVAL '30 days'`, filter `is_real = TRUE` on both, show the gap in days as `days_to_action`.

Hint: Use `BETWEEN ... AND ... + INTERVAL '30 days'` in the `JOIN` condition; compute `action_date - debate_date` for the gap.

Question 10 — Level 3

"Find all people who WALKED FREE despite being connected to organisations that also appear in the `banks` table from Chapter 3. JOIN `scapegoats_vs_escaped` with `banks` on `organisation = bank_name`."

Task: `JOIN scapegoats_vs_escaped` and `banks` on `organisation = bank_name`, filter `walked_free = TRUE`, show person name, role, organisation, reason escaped, bank type and bank involvement.

Hint: `JOIN` on a string match between two different column names.

Level 4 — The System on Trial

Subqueries, CTEs, Window Functions

Question 11 — Level 4

"Build a timeline of RBI actions using a window function. Show each action with a running COUNT of how many systemic changes had been made up to that point — and flag the first action and last action clearly."

Task: From `rbi_actions` where `is_real = TRUE`, use three window functions: `COUNT(*) OVER (ORDER BY action_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` for cumulative count, `FIRST_VALUE(action_type) OVER (ORDER BY action_date)` and `LAST_VALUE(action_type) OVER (ORDER BY action_date ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)`.

Hint: `LAST_VALUE` requires `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to see the true last row.

Question 12 — Level 4

"Rank every person in `scapegoats_vs_escaped` by how senior their role was — using a manual seniority score — and show whether the most senior people escaped or were punished."

Task: Use a CTE (`seniority`) that assigns a numeric rank via `CASE WHEN role_in_scam ILIKE '%Prime Minister%' THEN 1 ...` down to `ELSE 10`. In the outer query, apply a `CASE WHEN seniority_rank <= 3 AND walked_free = TRUE THEN 'HIGH POWER -- ESCAPED' ...` verdict label.

Hint: Build seniority in a CTE, then apply the verdict `CASE WHEN` in the outer `SELECT`.

Question 13 — Level 4

"Find all JPC witnesses who were hostile AND submitted evidence — the most dangerous witnesses to the establishment. Then check if any of them also appear in `politicians_questioned` using a subquery."

Task: From `jpc_hearings` where `is_real = TRUE`, `was_hostile = TRUE` and `evidence_submitted = TRUE`, add a `CASE WHEN witness_name IN (SELECT politician_name FROM politicians_questioned WHERE is_real = TRUE) THEN 'ALSO POLITICALLY QUESTIONED' ELSE 'Witness only' END` column.

Hint: Subquery inside `CASE WHEN ...IN (...)`.

Level 5 — The Final Reckoning

Multi-table CTE verdict query

Question 14 — The Political Storm Verdict

"Answer the one question the story keeps asking: Did the powerful escape while the small were punished? Combine five CTEs into a single verdict row — reforms passed, parliament motions, hostile witnesses, senior officials who escaped, and politicians never charged."

Task: Write a query using five CTEs:

- `reforms_passed` — `COUNT` of implemented RBI actions
- `motions_passed` — `COUNT` of real parliament motions passed

- `hostile_witnesses` — `COUNT` of hostile witnesses who gave evidence
- `senior_escaped` — `COUNT` of people from Government / RBI who walked free
- `politicians_never_charged` — `COUNT` of real politicians where `charges_filed = FALSE`

`CROSS JOIN` all five and add a final `CASE WHEN` verdict column.

Hint: Each CTE returns a single number. `CROSS JOIN` them all and use `CASE WHEN senior_escaped > 0 AND politicians_never_charged > 5` for the verdict.

Bonus — The Briefcase

The 1 crore question that was never answered

Bonus Question

Unscripted

"Harshad claimed he paid 1 crore to PM Narasimha Rao. Rao denied it. The JPC could not prove it. Pull every record across all Chapter 6 tables that mentions Narasimha Rao — hearings, debates, politicians questioned, scapegoats — and show the full picture of what the system recorded about this one claim."

Task: Write a `UNION ALL` query across four tables:

- `jpc_hearings` — where `witness_name ILIKE '%Narasimha Rao%'`
- `parliament_debates` — where subject, raised_by or govt_response mention Rao
- `politicians_questioned` — where `politician_name ILIKE '%Narasimha Rao%'`
- `scapegoats_vs_escaped` — where `person_name ILIKE '%Narasimha Rao%'`

Label each row with `table_source`, align column aliases (`event_date`, `person`, `record`), use `NULL` where a date column does not exist, and order by `table_source`, `event_date`.

Hint: Cast date columns with `::TEXT` to align types across `UNION ALL`; use `NULL` for missing date in the last branch.

Chapter 7

The Trial

SQL Concepts Covered: `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING`, `JOIN`, `CASE WHEN`, `IS NOT NULL`, *Subqueries, CTEs, Window Functions (*`COUNT OVER`, `RANK`*)*

Level 1 — The Docket Opens

Basic SELECT, WHERE, ORDER BY

Question 1 — Level 1

"The court clerk hands you a file. 'Here are all 27 real cases against Harshad.' List all real cases against Harshad Shantilal Mehta ordered by `filed_date` *— show case number, charge type and current status."*

Task: From `court_cases`, find all records where `is_real = TRUE` and `accused_name = 'Harshad Shantilal Mehta'`, showing `case_number`, `filed_date`, `charge_type`, `case_status` and `verdict`, ordered by `filed_date ASC`.

Hint: Two `WHERE` conditions joined by `AND`.

Question 2 — Level 1

"How many of those 27 cases ended in conviction, pending, acquittal or dropped? Group by `case_status` *from real Harshad cases."*

Task: From `court_cases` where `is_real = TRUE` and `accused_name = 'Harshad Shantilal Mehta'`, `GROUP BY case_status` and `COUNT(*)`, ordered by count DESC.

Hint: `GROUP BY case_status` + `COUNT` + `ORDER BY DESC`.

Question 3 — Level 1

"Walk the custody timeline. List all REAL custody events for Harshad ordered by date — this is the human story."

Task: From `timeline_of_custody`, find all records where `is_real = TRUE`, showing `event_date`, `event_type`, `location`, `description` and `harshad_condition`, ordered by `event_date ASC`.

Hint: `WHERE is_real = TRUE ORDER BY event_date ASC`.

Question 4 — Level 1

"How many times did Harshad apply for bail? From real bail records for Harshad — count how many were granted vs rejected."

Task: From `bail_records` where `is_real = TRUE` and `accused_name = 'Harshad Shantilal Mehta'`, `GROUP BY bail_granted` and `COUNT(*)`.

Hint: Boolean column as group key — `TRUE` = granted, `FALSE` = rejected.

Level 2 — Nine Years of Hearings

Aggregations, GROUP BY, HAVING

Question 5 — Level 2

"The defence lawyer wants a summary. For all real cases across ALL accused, group by `accused_name` and show how many cases each person faced, how many convictions, how many pending and how many acquittals."

Task: From `court_cases` where `is_real = TRUE`, `GROUP BY accused_name`, show `COUNT(*)` and four `SUM(CASE WHEN case_status = '...' THEN 1 ELSE 0 END)` columns for Convicted, Pending, Acquitted and Dropped, ordered by `total_cases DESC`.

Hint: Four separate `CASE WHEN` expressions inside `SUM`.

Question 6 — Level 2

"Which courts handled the most real cases? Group by `court_name` from real cases and count cases per court. Also show how many convictions each court delivered."

Task: From `court_cases` where `is_real = TRUE`, `GROUP BY court_name`, show `COUNT(*)` as `cases_heard` and `SUM(CASE WHEN case_status = 'Convicted' ...)` as `convictions`, ordered by `cases_heard DESC`.

Hint: `GROUP BY court_name` + `COUNT` + conditional `SUM`.

Question 7 — Level 2

"Analyse witness types in the real trial. Group by `witness_type` from real witnesses and count how many turned hostile vs stayed with their original testimony."

Task: From `witnesses` where `is_real = TRUE`, `GROUP BY witness_type`, show `COUNT(*)` as `total_witnesses`, `SUM(CASE WHEN turned_hostile = TRUE ...)` and `SUM(CASE WHEN turned_hostile = FALSE ...)`, ordered by `total_witnesses DESC`.

Hint: Two `SUM(CASE WHEN)` columns for hostile vs firm witnesses.

Level 3 — Building the Case

JOINs across tables

Question 8 — Level 3

"For each REAL convicted case, show the case details alongside the witnesses who testified in that case. JOIN `court_cases` and `witnesses` on `case_id`."

Task: `JOIN court_cases` and `witnesses` on `case_id`, filter `is_real = TRUE` on both tables and `case_status = 'Convicted'`, show case number, charge type, verdict, witness name, witness type, `turned_hostile` and testimony summary, ordered by case number then witness type.

Hint: Apply `is_real = TRUE` to both table aliases in `WHERE`.

Question 9 — Level 3

"Show every bail application alongside the case it was filed for. JOIN `bail_records` and `court_cases` on `case_id` for real records only — show the full back-and-forth between freedom and custody."

Task: `JOIN bail_records` and `court_cases` on `case_id`, filter `is_real = TRUE` on both, show bail applied date, granted status, amount, granted by, conditions, revoked flag, case number and charge type, ordered by `bail_applied_date ASC`.

Hint: `JOIN` on `case_id`; filter `is_real` on both aliases.

Question 10 — Level 3

"Connect appeals to their original cases. JOIN `appeals` and `court_cases` on `case_id` for real records — show what was appealed and what the original case was about."

Task: `JOIN appeals` and `court_cases` on `case_id`, filter `is_real = TRUE` on both, show appeal date, appellant, appeal court, grounds, outcome, case number, charge type and original case status, ordered by `appeal_date ASC`.

Hint: `JOIN` on `case_id`; alias `cc.case_status` as `original_case_status`.

Level 4 — The Long War

Subqueries, CTEs, Window Functions

Question 11 — Level 4

"Show the custody timeline with a running COUNT of how many events Harshad had faced by each date — and label the trial phase he was in using a `CASE WHEN` on the date."

Task: From `timeline_of_custody` where `is_real = TRUE`, use `COUNT(*) OVER (ORDER BY event_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` for cumulative events. Add a `CASE WHEN event_date < '1993-01-01' THEN 'Phase 1' ...` label for five phases up to death.

Hint: Window function for cumulative count; `CASE WHEN` on date ranges for phase labels.

Question 12 — Level 4

"Build a case-by-case damage report. For each REAL convicted case against Harshad, show the case, conviction date, and how many days it took from filing to verdict. Use a CTE and `RANK` by longest case duration."

Task: Use a CTE (`case_duration`) to compute `verdict_date - filed_date` as `days_to_verdict` for real convicted Harshad cases where `verdict_date IS NOT NULL`. In the outer query apply `RANK() OVER (ORDER BY days_to_verdict DESC)` as `slowest_justice_rank`.

Hint: Filter `verdict_date IS NOT NULL` in the CTE to avoid null arithmetic.

Question 13 — Level 4

"Find all cases that were still PENDING when Harshad died on December 31, 2001. These are the cases where justice was never delivered to either side. Use a subquery to get the death date from `timeline_of_custody`."

Task: From `court_cases` where `is_real = TRUE`, `case_status = 'Pending'` and `accused_name = 'Harshad Shantilal Mehta'`, add two subquery columns: the death date (`SELECT event_date FROM timeline_of_custody WHERE event_type = 'Death' ...LIMIT 1`) and the number of days the case was active (death date minus `filed_date`).

Hint: Same subquery used twice — once for the date column, once inside arithmetic. Consider aliasing it.

Level 5 — The Final Reckoning

Multi-table CTE verdict query

Question 14 — The Trial Verdict Query

"Build the complete account of 9 years of trial. One query. Nine years. One man. Total cases vs resolved vs pending at death, bail applications granted vs rejected, appeals resolved vs unresolved, witnesses turned hostile, and the last recorded event."

Task: Write a query using five CTEs:

- `case_summary` — counts by `case_status` for real Harshad cases
- `bail_summary` — total bail applications, granted vs rejected

- `appeal_summary` — total appeals, resolved vs unresolved at death
- `hostile_witnesses` — count of real witnesses where `turned_hostile = TRUE`
- `final_event` — latest record from `timeline_of_custody` using `ORDER BY event_date DESC LIMIT 1`

`CROSS JOIN` all five into a single summary row.

Hint: Each CTE is a single-row aggregate. `CROSS JOIN` collapses them into one row.

Bonus — December 31, 2001

The last day — pulling everything together

Bonus Question

Unscripted

"On December 31, 2001, Harshad Mehta died in Thane Civil Hospital. 23 cases still pending. Pull a final record: his first arrest date, his death date, how many days he fought the system, how many cases were filed vs resolved, and the very last custody event recorded."

Task: Write a query using three CTEs:

- `first_arrest` — `event_date` from `timeline_of_custody` where `event_type = 'First Arrest'`, `LIMIT 1`
- `death_day` — `event_date` and `description` where `event_type = 'Death'`, `LIMIT 1`
- `case_count` — total filed, resolved (`Convicted / Acquitted / Dropped`) and unresolved (`Pending`) for real Harshad cases

`CROSS JOIN` all three; compute `died_on - arrested_on` as `days_fought_the_system`.

Hint: Date subtraction works directly on `DATE` columns; `LIMIT 1` in each CTE ensures a single-row result for safe `CROSS JOIN`.

Chapter 8

The Comeback

SQL Concepts Covered: `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING`, `JOIN`, `CASE WHEN`, `EXISTS`, `NULLIF`, `EXTRACT`, `FILTER`, Subqueries, CTEs, Window Functions (`RANK`, `SUM OVER`)

Level 1 — The Bull is Back

Basic SELECT, WHERE, ORDER BY

Question 1 — Level 1

"It is 1997. Harshad is on bail. He launches `harshadmehta.com`. List all real predictions from his website ordered by date — show stock, predicted price, actual price and whether he was accurate."

Task: From `stock_predictions`, find all records where `is_real = TRUE` and `prediction_source = 'harshadmehta.com'`, showing `prediction_date`, `stock_name`, `predicted_price`, `actual_price`, `accuracy_pct` and `was_accurate`, ordered by `prediction_date ASC`.

Hint: Two `WHERE` conditions joined by `AND`.

Question 2 — Level 1

"List all REAL media interviews in order. Show what Harshad claimed, his tone and how the public reacted. This is the story of a man who never stopped fighting in public."

Task: From `media_interviews`, find all records where `is_real = TRUE`, showing `interview_date`, `publication`, `journalist`, `key_claim`, `harshad_tone`, `public_reaction` and `was_controversial`, ordered by `interview_date ASC`.

Hint: `WHERE is_real = TRUE ORDER BY interview_date ASC`.

Question 3 — Level 1

"Harshad kept trading through family accounts. List all real comeback trades ordered by date — show which account he routed through and whether he made profit or loss."

Task: From `comeback_trades`, find all records where `is_real = TRUE`, showing `trade_date`, `stock_name`, `trade_type`, `quantity`, `price_per_share`, `routed_through`, `profit_loss` and `is_direct`, ordered by `trade_date ASC`.

Hint: `WHERE is_real = TRUE ORDER BY trade_date ASC`.

Question 4 — Level 1

"List all real legal victories ordered by date. These are the moments the system had to give ground — even slightly."

Task: From `legal_victories`, find all records where `is_real = TRUE`, showing `victory_date`, `court_name`, `victory_type`, `description` and `significance`, ordered by `victory_date ASC`.

Hint: `WHERE is_real = TRUE ORDER BY victory_date ASC`.

Level 2 — Counting the Comeback

Aggregations, GROUP BY, HAVING

Question 5 — Level 2

"How accurate was Harshad's website overall? From real predictions with a non-null `was_accurate`, count how many were accurate vs inaccurate and calculate average accuracy percentage."

Task: From `stock_predictions` where `is_real = TRUE` and `was_accurate IS NOT NULL`, `GROUP BY was_accurate`, show `COUNT(*)` and `ROUND(AVG(accuracy_pct), 2)`, ordered by `was_accurate DESC`.

Hint: `IS NOT NULL` in `WHERE` to exclude unresolved predictions; boolean group key.

Question 6 — Level 2

"Which comeback trades were profitable vs loss-making? Group real trades by whether `profit_loss > 0` and sum total profit and total loss separately, with average per trade."

Task: From `comeback_trades` where `is_real = TRUE`, `GROUP BY` a `CASE WHEN profit_loss > 0 THEN 'Profit' WHEN profit_loss < 0 THEN 'Loss' ELSE 'Breakeven' END` expression, show `COUNT(*)`, `SUM(profit_loss)` and `ROUND(AVG(profit_loss), 2)`, ordered by total amount DESC.

Hint: The `CASE WHEN` expression must appear identically in both `SELECT` and `GROUP BY`.

Question 7 — Level 2

"The legacy table holds the final verdict on what Harshad left behind. Group by `legacy_type` and count how many positive vs negative legacies exist in each category — and flag the unanswered ones."

Task: From `legacy`, `GROUP BY legacy_type`, show `COUNT(*)` and three conditional `SUM(CASE WHEN is_positive = TRUE / FALSE / IS NULL ...)` columns for positive, negative and unanswered legacies, ordered by total DESC.

Hint: Use `is_positive IS NULL` (not `= NULL`) for the third `CASE WHEN`.

Level 3 — The Full Picture

JOINs across tables

Question 8 — Level 3

"For each real stock prediction that was accurate, check if Harshad also traded that same stock in `comeback_trades`. JOIN `stock_predictions` and `comeback_trades` on `stock_name` — show the prediction alongside the actual trade he made."

Task: `JOIN stock_predictions` and `comeback_trades` on `stock_name`, filter `is_real = TRUE` on both and `was_accurate = TRUE`, ordered by `prediction_-`

date.

Hint: JOIN on a string column (stock_name); apply is_real to both aliases.

Question 9 — Level 3

"Connect media_interviews to legal_victories by finding interviews that happened WITHIN 60 days of a legal victory — Harshad talking to the press right after a court win."

Task: JOIN legal_victories and media_interviews where mi.interview_date BETWEEN lv.victory_date AND lv.victory_date + INTERVAL '60 days', filter is_real = TRUE on both, show the gap as days_after_victory, ordered by victory_date.

Hint: Use BETWEEN ... AND ... + INTERVAL '60 days' in the JOIN condition; compute interview_date - victory_date for the gap.

Question 10 — Level 3

"The legacy table mentions SEBI, NSE, dematerialisation and circuit breakers — all born from the scam. JOIN legacy with rbi_actions from Chapter 6 on year implemented to show reforms that happened in the same year."

Task: JOIN legacy and rbi_actions where l.year_implemented = EXTRACT(YEAR FROM ra.action_date)::INT, filter l.is_positive = TRUE and ra.is_real = TRUE, ordered by year_implemented and legacy_type.

Hint: Cast EXTRACT(YEAR FROM ...) to INT to match the integer year_implemented column.

Level 4 — The Man Behind the Myth

Subqueries, CTEs, Window Functions

Question 11 — Level 4

"Build a yearly performance summary of Harshad's comeback trades using a window function. Show each year, total profit/loss that year, and a running total profit across all comeback years."

Task: From comeback_trades where is_real = TRUE and profit_loss IS NOT

NULL, GROUP BY EXTRACT(YEAR FROM trade_date), show yearly SUM(profit_loss) and a cumulative total using SUM(SUM(profit_loss)) OVER (ORDER BY EXTRACT(YEAR FROM trade_date) ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Hint: Nesting SUM inside a window SUM is valid when the inner SUM is the result of GROUP BY.

Question 12 — Level 4

"Rank all comeback era stocks by total profit generated across all trades. Use RANK() and show whether the stock was also predicted on the website."

Task: Use a CTE (stock_profit) to SUM (profit_loss) per stock_name from real comeback trades. In the outer query apply RANK() OVER (ORDER BY total_profit DESC) and a CASE WHEN stock_name IN (SELECT stock_name FROM stock_predictions WHERE is_real = TRUE AND prediction_source = 'harshadmehta.com') THEN ... label.

Hint: Subquery inside CASE WHEN ...IN (...) for the prediction status check.

Question 13 — Level 4

"Find the THREE most significant legal victories based on a manual significance scoring — and show whether each was followed by a media interview within 90 days."

Task: Use a CTE (scored_victories) that assigns a numeric score via CASE WHEN victory_type ILIKE '%Bail%' THEN 3 ...up to 5 for dropped/abated cases. In the outer query add a CASE WHEN EXISTS (SELECT 1 FROM media_interviews WHERE interview_date BETWEEN sv.victory_date AND sv.victory_date + INTERVAL '90 days' AND is_real = TRUE) THEN 'Yes ...' ELSE 'No ...' END column. LIMIT 3.

Hint: EXISTS is more efficient than IN for correlated subqueries; ILIKE for case-insensitive matching.

Level 5 — The Final Reckoning

The complete Harshad story — all 8 chapters converge

Question 14

The Game's Final Verdict

"Everything the game has built across 8 chapters comes down to this one query. His peak trade year, his website accuracy, his court victories, his legacy balance, his most defiant interview claim, and his final custody event. One query. One man. One story."

Task: Write a query using six CTEs:

- `peak_trade_year` — best year by `SUM(profit_loss)` from `comeback_trades`, `LIMIT 1`
- `website_accuracy` — total predictions, accurate count and `ROUND` accuracy rate using `NULLIF` to avoid division by zero
- `total_victories` — `COUNT(*)` from `legal_victories` where `is_real = TRUE`
- `legacy_balance` — positive, negative and unanswered counts from `legacy`
- `most_defiant_claim` — earliest interview where `harshad_tone = 'Defiant'`, `LIMIT 1`
- `final_moment` — last event from `timeline_of_custody` using `ORDER BY event_date DESC LIMIT 1`

`CROSS JOIN` all six into a single summary row.

Hint: Use `NULLIF(COUNT(*) FILTER (WHERE was_accurate IS NOT NULL), 0)` as the denominator in the accuracy rate to safely handle nulls.

Bonus — The Unanswered Question

The last query of the entire game

Bonus Question

The Last Query

"The legacy table has one row where `is_positive IS NULL`. That is the philosophical row — the question the game has been asking since Chapter 1. Pull that row. Then pull the first event from `timeline_of_custody` (his first day in Bombay) and the last event (his death). Three rows. The beginning, the question, and the end."

Task: Write a `UNION ALL` query across three parts:

- **The Beginning** — earliest real record from `timeline_of_custody` using `ORDER BY event_date ASC LIMIT 1`; cast `event_date::TEXT` as the date

column

- **The Unanswered Question** — record from `legacy` where `is_positive IS NULL`; cast `year_implemented::TEXT` as the date column
- **The End** — latest real record from `timeline_of_custody` using `ORDER BY event_date DESC LIMIT 1`; cast `event_date::TEXT` as the date column

Label each row with a `moment` column. Align four aliases: `moment`, `date`, `what`, `detail`.

Hint: All three `SELECT`s must have the same column count and compatible types. Cast date and integer columns to `TEXT` so `UNION ALL` can align them.

The data has spoken.

8 Chapters. 40+ Tables. 100+ SQL Puzzles.

From Raipur to Rajya Sabha — one query at a time.

You started with a `SELECT *`.
You ended with a `CROSS JOIN` across five CTEs
and a man's entire life.

That is not just SQL.

That is thinking like an analyst.

8	chapters of real history
40+	tables of structured truth
100+	SQL puzzles
1	story that still has no clean verdict

Every query you wrote was a question.
Every result set was an answer the system
either recorded — or tried to bury.
You learned to find both.

Now go build something real.
The database is waiting.

Prabeen Ragupathi